

오늘의 작업기록 — 2026년 6월 30일

mini-ledger Layer 1 — 이력 조회(특정 계좌 거래 필터링)

(5단계를 처음 혼자 몰아본 날 — 그리고 시간 없다고 넘기지 않고 끝까지 뜯은 날)

한눈에

항목	내용
목표	특정 계좌의 거래만 뽑는 조회 기능 (환불의 온램프)
결과	완료 ✓ — a 조회 시 a 킨 3건만(WITHDRAW b 빠짐) 출력 확인
커밋	add useHistoryByAccount for transaction filtering
저장소	github.com/Taewoo-Won/mini-ledger (7개 .java)
오늘의 핵심	5단계 첫 솔로 런 + ②에서 "Bank냐 Main이냐" 스스로 교정
새 개념	필터링 공식 · <code>.equals()</code> vs <code>==</code> · 매개변수=빈칸 · <code>return</code> ≠ 출력
제약	파견 · 폰 회수 가능성으로 시간 압박 → 잔디 먼저, 그 후 코드 완전 해체
학습 방식	5칸 직접 채움 → 막힌 ②를 스스로 뚫음. 이해 안 된 4가지를 끝까지 질문해 청산

시간이 없어서 잔디만 박고 넘어갈 뻔한 날. 근데 "시간 없어 넘어감" 문구를 남기지 않으려고, 잔디 후 오늘 코드를 통째로 뜯어 온전히 내 것으로 만들었다. 그 과정에서 매 개변수·equals·return의 본질을 처음 제대로 잡았다.

0. 출발점 / 오늘의 목표

- 이전까지: 증분 1~5 완료 (Java 기초 + Layer 1 절반: 감사 로그·역등성).
- 오늘 목표: 이력 조회 — 특정 계좌(예: a) 거래만 필터. 첫 솔로 런(5단계 직접 몰기).
- 왜 이것: Layer 1의 정산·환불로 가는 온램프. 환불하려면 원본 거래를 찾아야 하고, 그게 "조회"다. (시간 제약상 가벼운 이걸로 — 순서상으로도 먼저 와야 함.)
- 다음: 정산 / 환불 (Layer 1 나머지 절반). 시간 여유 있는 세션에.
- 작업 도구: 폰 + VPS + nano + javac. (파견 중, 폰 회수 가능성 → 잔디 우선 전략.)

PART 1 — 무엇을 / 왜 (도메인)

지금 useHistory() 는 전체 거래를 다 준다. 근데 실무에선 "내 통장 내역만" 보고 싶다 → 특정 계좌 거래만 거르는 기능이 필요하다.

이게 환불의 전제다: 환불하려면 "어떤 거래를 환불할지" 원본을 찾아야 하고, 찾으려면 조회가 있어야 한다. 그래서 정산·환불 전에 조회가 먼저다.

PART 2 — 5단계 솔로 런 (오늘의 본체)

처음으로 5칸을 혼자 채웠다. 결과:

단계	내 답	평가
① 데이터	없음 (history 재사용, 새 필드 X)	✓ 정확
② 입구	~~Main~~ → Bank에 useHistoryByAccount(id)	★ 교정 (아래)
③ 로직	"읽어서 거른다" = 실행/기록 쪽, 검증 X	✓ 정확
④ 호출	Main에서, 전체 출력 for 아래	✓
⑤ 확인	a 킨 것만 전체보다 적게 나오면 성공	✓

★ ②의 교정 — 오늘 제일 큰 수확

처음엔 "구체적인 거 호출은 Main 역할이니까 메서드도 Main"이라 생각했다. 틀렸다.

- history는 Bank의 private → Main에서 직접 못 만진다(못 본다).
- → "history를 거르는 작업"은 history가 있는 Bank 안에서 해야 한다.
- 메서드(거르는 로직)는 Bank에, 호출(부탁)은 Main에서. 둘은 다르다.

핵심 원칙(스스로 도달): "데이터가 사는 곳에서 그 데이터를 다룬다." 28일 useHistory를 Bank에 만든 것과 같은 이유. 막혔다가 질문으로 뚫었다.

PART 3 — `getHistoryByAccount` 완전 해체 (필터링 공식)

```
public List<Transaction> getHistoryByAccount(String id) {
    List<Transaction> result = new ArrayList<>();
    for (Transaction t : history) {
        if (id.equals(t.getFrom()) || id.equals(t.getTo())) {
            result.add(t);
        }
    }
    return result;
}
```

이건 백엔드 필터링의 표준 형태. 5조각:

조각	코드	역할
시그니처	<code>List<Transaction> ... (String id)</code>	id 받아서 거래 목록 반환 (반환 타입=목록, <code>getHistory</code> 와 동일)
빈 바구니	<code>List result = new ArrayList<>()</code>	거른 것 담을 새 목록. history 는 원본 보존
순회	<code>for (Transaction t : history)</code>	전체 영수증 한 장씩
조건	<code>if (id.equals(from) id.equals(to))</code>	그 계좌가 낀 거래면
담기	<code>result.add(t)</code>	바구니에 추가
반환	<code>return result</code>	다 돌고 바구니 통째로

필터링 공식 = 빈 바구니 → 전체 순회 → **if**로 거르기 → **add**로 담기 → **반환**. 조건만 바꾸면 뭐든 거른다(금액 1000 ↑ 만, DEPOSIT만 등).

3-1. 조건 `id.equals(from) || id.equals(to)` 정밀 해부

내가 헛갈렸던 것: "from과 to가 같아야 하나?" → **아니다**.

- `id.equals(t.getFrom())` = "점 쪽(**id**)과 괄호 안(**from**)이 같냐" → **id와 from** 비교
- `||` = 또는
- `id.equals(t.getTo())` = **id와 to** 비교
- **from↔to**를 비교하는 게 아니라, **id↔from** / **id↔to** 각각 비교. 둘 중 하나만 맞아도 "그 계좌가 낀 거래."

값 대입 (id="a"):

```
TRANSFER a→b: "a".equals("a")=T || ... = T → 답음 (a가 보냄)
DEPOSIT →a:    ... || "a".equals("a")=T = T → 답음 (a가 받음)
WITHDRAW b→null: "a".equals("b")=F || "a".equals(null)=F = F → 안 답음 ✓
```

→ "a의 거래" = a가 보냈거나(from=a) a가 받은(to=a) 것 둘 다. 그래서 OR.

PART 4 — Main 호출

```
System.out.println("===== a 계좌 거래만 =====");
for (Transaction t : bank.getHistoryByAccount("a")) {
    System.out.println(t.getType() + " " + t.getFrom() + " -> " + t.getTo() + " : " + t.getAmount());
}
```

- 기존 전체 출력 for문과 거의 동일. 차이는 `getHistory()` → `getHistoryByAccount("a")` 하나.
- 출력 방식은 같고 데이터만 **a로 걸러진 것**.

실행 결과 (⑤ 검증)

```
===== 거래 내역 =====      (전체 4건)
TRANSFER a -> b : 200
TRANSFER a -> b : 1000
DEPOSIT null -> a : 500
WITHDRAW b -> null : 300          ← b만 낀 거래

===== a 계좌 거래만 =====      (a 낀 3건)
TRANSFER a -> b : 200
TRANSFER a -> b : 1000
DEPOSIT null -> a : 500          ← WITHDRAW b 빠짐 ✓
```

→ `WITHDRAW b` 가 정확히 빠짐. 필터 작동 확인.

PART 5 — "왜?" 문답 (끝까지 청산한 4가지)

1. 코드엔 "a"가 없는데 왜 "a와 비교"인가? 메서드 안 `id` 는 빈칸(매개변수). 코드 자체엔 a가 없다. Main에서 `getHistoryByAccount("a")` 로 부를 때 그 빈칸에 "a"가 채워져서 실행 중 `id="a"`가 된다. "b" 넣으면 b 거래를 뽑는다. → "a"는 메서드의 일부가 아니라, 부를 때 바깥에서 꽂는 값. (`deposit("a", 500)`에서 `id`·`amount` 채워지던 것과 동일.)
2. `.equals()` 는 뭘 비교하나? `from↔to`인가? 점 쪽 ↔ 괄호 안. 여기서 `id↔from`, `id↔to`. `from`과 `to`끼리는 안 본다. 각각 `id`와 비교해서 OR.
3. `.equals()` vs `==` — 왜 `equals`? 문자열(String) 같은지는 반드시 `.equals()` . `==` 는 "같은 메모리 자리냐", `.equals()` 는 "내용(글자)이 같냐"를 본다. 문자열은 내용이 중요 → `equals`. 규칙: 문자열= `.equals()` , 숫자(long)= `==` . (잔액 비교는 `==` 맞았음.)
4. 조건 맞으면 `return`이 발동? 그때 출력되며 뽑히나? 아니다. 두 가지 정정 — 조건 맞을 때 발동하는 건 `result.add` (담기, 여러 번). `return` 은 for 다 끝난 후 한 번. - `return` 은 출력이 아니라 "목록을 Main에 건네기". 화면 출력은 건네받은 Main의 `for+println`이 한다. - → `거르기(Bank add)` → `건네기(return)` → `출력(Main println)` 3단계.

제일 중요한 건 1·4. 매개변수=빈칸(1)과 `return`≠출력(4)은 모든 메서드에 통하는 본질이다.

6. 산출물

- 파일: `Bank.java` (`getHistoryByAccount` 추가), `Main.java` (a 조회 출력 추가). 나머지 변경 없음.
- 검증: a 조회 → a 킨 3건만(WITHDRAW b 제외).
- 커밋 계보: `...403ef81` (데이터모델) → `fb3897b` (감사로그) → `역등성` → `getHistoryByAccount` (조회)

7. 다음 작업 (Layer 1 나머지)

- 환불 — 원본 거래를 (오늘 만든 조회로) 찾아 반대 거래 생성. 오늘 조회가 그 전제.
- 정산 — 이체에 수수료 분배.
- 복식부기 — 모든 거래가 양쪽을 건드리게. → 시간 여유 있는 세션에 하나로 스코프.

8. 개념 사전

- 필터링 공식: 빈 바구니(새 List) → 전체 순회(for) → 조건(if) 맞으면 담기(add) → 반환(return). 조건만 바뀌 재사용.
- 매개변수 = 빈칸: 메서드 정의의 `id` 등은 빈칸. 호출 때 값이 채워짐. 한 메서드로 `a·b·c` 다 처리.
- `.equals()` : 문자열 내용 비교. 점 쪽과 괄호 안을 비교. (vs `==` = 메모리 동일성.)
- `==` vs `.equals()` : 숫자·boolean은 `==` , 문자열·객체 내용은 `.equals()` .
- `||` (OR): 둘 중 하나라도 참이면 참.
- `return`: 값을 호출한 쪽에 건네기(≠ 출력). 출력은 받은 쪽이 `println`으로.
- 원본 보존: history를 직접 거르지 않고 새 `result`에 담아 반환 → 원본 목록 안전.

9. 비유 모음

- 필터링 = 영수증 골라 봉투에 담기: 전체 더미(history) 그대로 두고, a짜리만 골라 빈 봉투(result)에 담아 건넨다.
- 매개변수 빈칸 = 자판기 버튼: `getHistoryByAccount(?)` 는 "어느 계좌?" 버튼. "a" 누르면 a, "b" 누르면 b. 기계(메서드)는 하나.
- `return` ≠ 출력 = 봉투 건네기 vs 읽기: Bank가 봉투를 건넨(return). 펼쳐 소리내 읽는 건(println) 받은 Main.
- `equals` vs `==` = 내용 vs 자리: 쌍둥이 두 명 — "같은 사람이나"(==, 한 사람인가)와 "똑같이 생겼냐"(equals, 내용 같은가)는 다르다. 문자열은 "똑같이 생겼냐".

10. 검증 질문 (다음날 복기용 — 답 적지 말 것)

- 이력 조회 메서드를 Main이 아니라 Bank에 만드는 이유는?
- 필터링 공식 4단계를 순서대로 말하라 (바구니→?→?→반환).
- 메서드 코드 안엔 "a"가 없는데 왜 실행 중 "a와 비교"가 되냐?
- `id.equals(t.getFrom())` 은 무엇과 무엇을 비교하나? `from`과 `to`를 비교하는 게 아닌 이유?
- 문자열 비교에 `==` 대신 `.equals()` 를 쓰는 이유는?
- `return result` 는 출력인가? 화면 출력은 실제로 어디서 일어나나?

11. 회고

오늘은 "넘어갈 뻔한 날"이었다. 파견에 폰 회수 가능성까지, 시간이 없었다. 그래서 순서를 바꿔 **잔디부터** 박았다 — 옳은 판단이었다(완벽한 작업기록보다 끊기지 않는 기록이 우선). 근데 거기서 멈추지 않고, "시간 없어 넘어감"을 남기지 않으려 **코드를 통째로 뜯었다**. 그 30분이 오늘을 살렸다.

수확 들. ① **첫 솔로 런** — 5칸을 혼자 채웠고, ②에서 "Main이냐 Bank냐"를 틀렸다가 "history가 private이니 Bank"로 스스로 교정했다. 답을 받은 게 아니라 막힌 걸 뚫은 거라 값지다. ② **본질 4개를 끝까지 캐다** — 매개변수가 빈칸이라는 것, equals가 점↔괄호를 비교한다는 것, equals와 ==의 차이, return이 출력이 아니라 건네기라는 것. 특히

"코드에 a가 없는데 왜 a 비교냐"는 매개변수의 핵심을 정확히 찌른 질문이었다.

필터링 공식(바구니→순회→if→반환)은 조회·검색·집계 어디서나 쓴다. 오늘 이걸 손에 넣었다. Layer 1 환불로 가는 길의 첫 돌을 놓았고, 무엇보다 "시간 없어도 대충 안 넘긴다"를 지켰다.

한 줄: 시간에 쫓겨도 잔디부터 지키고 코드는 끝까지 뜯었다 — 첫 솔로 런으로 필터링 공식을 손에 넣고, 매개변수·equals·return의 본질을 온전히 내 것으로 만든 날.